tenancy.

- When a user submits a query, the system first *embeds* it, converting the text into a numerical vector that represents its semantic meaning.
- The system uses the query embedding to retrieve the most relevant chunks from the vector database.
- Those retrieved chunks are inserted into the prompt as additional context.
- The language model generates a response using both the user's instruction and the retrieved material.

This pattern is well documented and widely implemented; canonical descriptions can be found in documentation from OpenAI, Amazon Web Services, and LangChain. From an authorization perspective, however, one detail matters more than all others: RAG adds system-selected context to the user-provided context. The RAG application determines which information is retrieved and presented to the AI. That decision point is exactly where authorization must be enforced.

17.4.2 Retrieval is the enforcement boundary

In traditional applications, authorization controls what data a requester can access directly. In a RAG system, retrieval serves a similar purpose. Retrieved documents become part of the model's working context, influencing not only what it can state explicitly but also what it can infer implicitly when generating a response.

Many early RAG implementations tried to control access at the prompt or response level. Prompts contained instructions like "do not reveal sensitive information," or responses were edited afterward to remove unwanted content. These methods view the language model as a tool for enforcement.

ACME considered this approach to be inherently unsafe. Language models infer intent from context, respond based on probabilities, and lack an understanding of policy, scope, or accountability. Once unauthorized data enters the model's context window, it can subtly influence the output in ways that are hard to detect or audit, even if the final response seems compliant.